

Combined MPLPB

A Provenance Ledger with a Refusal: One Format, One Rule, and What the Kill Test Measured

Field	Value
Document ID	MPLPB-COMBINED-017 (proposed; the next free number after MPLPB-IMAGE-016 in the supplied material, not yet allocated by the owner's index)
Category	Specification / Reference Implementation
Subcategory	Knowledge Infrastructure / Consolidation
Updated	2026-10-07T00:00Z v1
Owner	Mitchell D. McPhetridge, Independent Researcher
Status	draft for owner review
Scope	The single record format and the single answer rule that the MPLPB papers and implementations share; a standard-library reference implementation of exactly that rule; how each earlier part maps onto it; and one run of the kill test with its limits
When to use	Starting a new corpus and wanting one format instead of five; reading a corpus written by any earlier MPLPB tool; deciding what the formula does and does not guarantee; checking what the kill test showed
Supersedes	Nothing. Every earlier paper and implementation stays current within its own scope and is carried under parts/

Related. MPLPB as a Local Web (MPLPB-LOCAL-008 v4). Separate, Allowed to Chat, and Overridden by Scope (MPLPB-SEP-007). Smart Local MPLPB (MPLPB-SMART-011 v1). MPLPB and the Cost of Visibility (MPLPB-COST-012 v2). MPLPB at Swarm and Enterprise Scale (MPLPB-SWARM-013 v3). Networked MPLPB. MPLPB Image (MPLPB-IMAGE-016 v2). Multi-Platform Linked Public Building (v3) and the other chapters of *Continuity Without Memory*.

Implementation. This repository. Package `mplpb_combined`, 93 tests, standard library only, no network, no model, no database.

Back to. [Main Index](#) > [Specification / Architecture Sub-Index](#)

Abstract

The MPLPB papers describe one idea several times at several scales: a body of knowledge kept as complete pages that declare what they are, read by something that is not allowed to answer from anywhere else. By the time the idea had a local form, a reader, a cost argument, a swarm form, a networked form and an image form, it also had five implementations, three spellings of the same metadata, and two different procedures for retiring a page.

This paper states the part they have in common and nothing else. It is a closed local ledger of pages that answers only from those pages. Each page carries its own id, scope, status, hash, parent and depth. The reader crawls the folder. If one current page owns the question, it returns that page. If two own it, it stops.

If none own it, it says not in the corpus. A change writes a new page and retires the old one. A derivation writes a new page and adds one to the depth.

The reference implementation is about twelve hundred lines of standard-library Python for the format, the ledger and the reader, plus a command line and the test harness. It reads the example sites of four earlier implementations after adding hashes to them, and it carries all five implementations unchanged in behaviour under parts/.

The formula proposed a proof: strip the scope fields, run the same questions, and show the refusal disappears. That test has now been run once, on one small corpus, by the same hand that wrote the corpus. The result is mixed and is reported as it came out. The reader with declared scope returned no wrong page on 56 questions, where a plain search over the same pages with the fields stripped returned 36. It also refused all 8 questions that were phrased from a page's prose instead of its declared words, which the stripped arms answered correctly. And the same three-case rule applied to prose did not lose its refusal; it refused almost everything. The refusal is not what the declared fields supply. What they supply is something for the refusal to be decided from.

1. The Formula

A record is

```
R = (id, scope, status, hash, derived_from, origin_depth)
```

Status is current or retired. A question q is answered only from current records whose scope owns q :

```
A(q) = R          if exactly one current R owns q
      ambiguous    if two or more own q
      not in corpus if none own q
```

No fill from outside the folder. No average of two owners.

Derivation does not copy the hash. It writes a new record and increments depth:

```
R[n+1].derived_from = R[n].id
R[n+1].origin_depth = R[n].origin_depth + 1
```

A human source starts at depth 0. A revision retires $R[n]$; it does not overwrite it. A profile may refuse any record with depth above a set limit. A hub may point at the owner; it may not answer as the owner.

Existing parts, one composition rule: do not blend, do not erase, count the generations.

Everything below is what had to be decided to make those sentences executable, and each decision is marked as one.

2. The Format

A record is one HTML file. Its `<head>` carries `<meta name="mplpb:...">` tags and its `<body>` carries the words. Opened in a browser it is a page. Nothing else is needed to read it.

Field	Required	Meaning
document-id	yes	Allocated, never chosen, never reused
scope	yes	One sentence saying what the page covers
status	yes	current or retired
hash	yes	SHA-256 over the title, the body, and every field except status and hash
derived-from	when derived	Parents, each as ID@sha256: . . .
origin-depth	yes	Generations between this page and a source
when-to-use	no	The situations in which someone should land here, in the asker's words
supersedes	when revising	The page this one replaces, as ID@sha256: . . .
origin	no	human or machine; absent means human
ratified-by	when ratified	The name of the person who stands behind a derived page
kind, points-to	hubs only	pointer, and the folder it points at
owner, updated, category	no	Carried, hashed, not interpreted

Three decisions in that table are new and deliberate.

The hash excludes status. Retiring a page changes one line in its head. Because that line is outside the hash, a retired page still verifies, and "it does not overwrite it" becomes checkable: the hash of the retired page is the hash it had when it was current.

References are pinned. A child names its parent's id and its parent's hash. Editing a parent, or swapping in a different page under the same id, breaks every descendant's pin. This is integrity without keys. It shows a page was not changed after it was written. It does not show who wrote it; signatures are the Networked part's job and stay there.

One spelling. The earlier tools wrote `mplpb:document-id` and `mplpb:id`, `origin-depth` and `origin_depth`, `updated` and `version`. All are read as the same field. New pages are written with hyphens.

A file under the root with no `document-id` is not a record and is never read. An index page is recognised by name, category or kind and never owns a question. `index.html` is rebuilt from the records for people to browse and carries no authority.

3. What "Owns" Means

The formula leaves one word undefined, and it is the word everything depends on. The earlier reader, Smart Local, defines ownership as a ranked full-text score in which one spoke leads the runner-up by 1.5 times, and calls that number the most arbitrary in the package. This implementation defines ownership without a score.

- 1 A page **declares** the content words of its scope and when-to-use fields. Content words are lower-cased, stripped of a short stop list, and folded for a trailing plural s. There is no stemmer and no synonym table.
- 2 A question's content words that some current page declares are its **known** words.
- 3 A page **owns** the question when it declares every known word.

Then the rule is the formula, with one clarification for the case it does not name:

- exactly one owner: return it;
- two or more owners: ambiguous;
- no owner, but two or more pages each declare part of the question: ambiguous, because answering would mean assembling one answer from two pages;
- no page declares any word of it: not in corpus.

Page prose is what is returned. It is never what is matched. That is the whole difference between this and a search engine, and section 8 measures what it costs.

Two constants remain, and both are chosen, not derived. A word declared by more than half the pages grants no claim, once there are four pages, because a word every page declares cannot tell them apart. And a profile sets a **coverage**: the share of a question's content words that must be known before any page may own it. Coverage is the reach-against-precision dial that MPLPB-SWARM-013 section 10.3 measured for its ranker. On the corpus tested here it turned out to matter very little (section 8.4).

The decision function is under twenty lines, takes two sets, and is shared by the reader and by the kill test's control arm, so the two cannot drift apart.

4. Writing

Four verbs write to a ledger. None of them changes a word of an existing page.

write allocates the next id for a prefix, renders the page with its hash, and puts it in the folder. It refuses if the file exists.

revise writes a new page that names the old one in *supersedes*, then sets the old page's status line to *retired*. The order is the point. The reader treats any page named in a *supersedes* field as retired whatever its own status line says, so a crash between the two steps leaves a ledger that still reads correctly, and the validator reports the unfinished step. Smart Local's notebook retires first and writes second; a crash there leaves no current page, and the reader says "not in this corpus" about something that exists.

Retired pages are not moved. The earlier tools move them to `_log/superseded/` and treat a retired page elsewhere as a fault. Here location carries no meaning and status does, which removes a step that can fail and a class of broken relative links.

derive writes a new page naming its parents, pinned, and sets depth to one more than the deepest of them.

ratify is a revision that changes no words. It writes a new page with the same body and lineage, the name of the person standing behind it, and depth 0. The origin stays *machine*. The citation then reads *origin*

machine d0 · ratified by Name, which says more than rewriting the origin to human would.

Writers take a lock file for the duration of one write. A lock older than thirty seconds belonged to a writer that died. This is enough for a person and a few scripts. It is not the lease-and-fencing discipline of MPLPB-SWARM-013 section 4, which remains the answer when writers are many.

Every write and every retirement appends a line to `_log/ledger.jsonl`. Each line carries the hash of the line before it, so the log cannot be edited in the middle without the chain breaking.

5. Depth

The formula says a derivation adds one. Five cases needed settling.

Case	Depth
Human page with no parents	0
Machine page with no parents	1. The loop cannot award itself a human signature (MPLPB-SWARM-013 section 5.3)
Derived page, whoever derives it	1 + the deepest parent
Revision	At least the depth of the page it replaces
Ratified page	0, with the ratifier's name on the page

The fourth row closes a path the earlier tools leave open. If a revision took its depth only from its author, a person tidying a depth-3 page would produce a depth-0 page without ever saying they stood behind it. Here a reset happens only through `ratified-by`, which is a name on the page and a line in the log.

The reader does not trust the declared number. It recomputes depth from the lineage and serves the larger of the two. A page that understates its depth is an error to the validator and is not believed by the reader.

What this does not do is unchanged from the swarm paper: it does not detect an invented claim. A writer that marks a machine page human, or types a name into `ratified-by` without reading, defeats it. Depth is a count of declared generations.

6. Profiles and Hubs

A profile is two numbers: the deepest record it will serve, and the coverage a question needs.

Profile	Max depth	Coverage
lab	2	34%
internal (default)	1	50%
external	0	100%

A page over the limit is removed before ownership is decided. When that changes the outcome, the refusal says so and gives the page's id and depth. `answer_without_retrieval` has no setting here because there is no code path that produces words other than a page's own.

A hub is a ledger whose records are pointers. A pointer declares a scope like any page and names another folder. When a pointer owns a question, the reader asks the same question in that folder and returns what that folder returns, with the route attached. When no pointer declares the question, each folder the hub points at is asked in turn; exactly one may answer, and two answering is ambiguity between corpora, which the hub names and stops on. A folder that cannot be reached produces a refusal that names it. No path returns a pointer's own body, and no path merges two folders' answers. This is MPLPB-SEP-007 one level up, as the swarm paper specified, without the network that Networked MPLPB adds.

7. How the Parts Map

Nothing is superseded. Each part keeps what only it does.

Part	What the combined core takes from it	What stays in the part
Public MPLPB (v3) and the complete-artifact header	The fields: identity, scope, trigger conditions, status, supersession	The public web as substrate; reconstruction by a search-capable assistant; the recoverability, stability and validation questions
Rabbit Hole Internet Indexer	Nothing executable. The core has no index	An indexing and retrieval layer in front of a corpus, with embeddings as a replaceable component
Local Web (LOCAL-008) and the Local Mirror	Pages as plain HTML under one root; bounded reading; metadata in the head	Link-graph crawl, typed links, the eight structural checks, the boot block
Separation and Precedence (SEP-007)	The rule itself: scope decides, ties stop, nothing merges	The argument for it
Mode controller (MODE-008)	Nothing. The core has one behaviour	Modes, overrides, the hard-reality priority
Smart Local (SMART-011)	Refusal as the default; provenance as part of the hit; revision as supersession	FTS5 ranking, the 1.5× margin, the console, the browser view, teaching
Cost of Visibility (COST-012)	The constraint: consumer hardware, no service	The cost model and the comparability claim
Swarm (SWARM-013)	Origin depth, ratification, profiles, allocated ids, routing between corpora without merging	Leases with fencing tokens, the registry, health metrics, the graded ablation
Networked MPLPB	Hubs that point and do not answer; unavailable is never impersonated	Ed25519 signatures, nodes, mirrors, transport
MPLPB Image (IMAGE-016)	The principle that the page is the control object and the file is the artifact	Pixel evidence, attestation, the learner
Public Experiment (TEST-001, TEST-002)	Pass criteria fixed before a run	The replication protocol and trial ledger

On search and the public web. The formula is a property of a closed reader. A folder read by this package obeys it. The same pages published to the open web are read by somebody else's search engine and somebody else's model, and nothing stops either from blending two pages or filling a gap. Public MPLPB can make the right answer findable and can make the fields travel with it. It cannot enforce the

refusal. That limit follows from the formula's first word, and the public papers already separate recoverability from validation for the same reason.

A ranker, lexical or semantic, can still sit in front of this core to propose candidates on a large corpus. It may narrow which pages are considered. It may not make the decision.

8. The Kill Test

8.1 What was run

One corpus, examples/studio: 25 pages about running a pottery studio. 21 are current and 4 are retired. Three of the current pages are machine-written: one at depth 1, one at depth 2, and one ratified back to depth 0. 56 questions in eight classes, with the right outcome for each fixed in a file whose hash was recorded before the first run.

Three arms read the same pages.

- **scoped** is the reader in this package at the internal profile.
- **stripped** applies the identical three-case rule after every `mplpb` field is removed. With no declarations left, ownership is decided from page prose, and with no status left, retired pages cannot be told from current ones.
- **top1** is what a plain search does with the stripped pages: return the page sharing the most words with the question, and refuse only when nothing shares any. It has no way to say ambiguous.

Scoring is mechanical. An answer is correct when the outcome matches and, for a return, the page matches.

8.2 Results

Class	n	scoped	stripped	top1	Note
Asked in a page's declared words	10	100.0%	10.0%	20.0%	
Asked from a page's prose	8	0.0%	100.0%	100.0%	control at ceiling
Declared situation plus unfamiliar words	8	62.5%	0.0%	12.5%	
Two current pages both answer	6	100.0%	50.0%	0.0%	circular for top1
The answer was revised	6	100.0%	0.0%	16.7%	circular for both stripped arms
Only owner is over the depth limit	2	100.0%	0.0%	0.0%	circular for both stripped arms
Nothing to do with the corpus	6	100.0%	100.0%	100.0%	control at ceiling
Studio question no page covers	10	60.0%	40.0%	0.0%	
All 56	56	73.2%	39.3%	32.1%	
Non-circular classes only	42	64.3%	45.2%	40.5%	

How each arm was wrong matters more than how often.

Arm	Wrong page or false return	False or misplaced refusal
scoped	0	15
stripped	3	31
top1	36	2

8.3 Reading it

The reader with declared scope returned no wrong page. Fifteen times it refused when it should not have, or refused in the wrong way. It never handed back a page that was not the answer. Plain search over the same pages returned a page for 48 of the 56 questions and was wrong on 36 of them, including all ten studio questions that no page covers. Smart Local's design rests on the claim that an occasional "I don't have that" is cheap and an occasional confident wrong answer is corrosive. On this run the declared fields bought exactly that trade.

The refusal did not disappear when the fields were stripped. It lost its footing. The stripped arm uses the same rule and refused 31 times it should not have. Ordinary words recur across prose, so nearly every question had several part-owners and the rule stopped. The proof the formula proposed, "strip the scope fields and show the refusal disappears", holds only against a reader that has no refusal to begin with, which is the top1 arm, and there it is partly circular. The more exact statement is that the rule needs something small and deliberate to be decided from, and declared scope is that thing.

Pages are reachable only by the words their author predicted. All eight questions phrased from a page's prose were refused by the scoped reader and answered by both stripped arms. "How long do I candle the burners" has an answer in the corpus and the reader said it did not. This is the price of never matching prose, it is paid in full, and a when-to-use field that misses how people ask makes a page invisible.

Incidental words cause most of the misplaced refusals. On uncovered studio questions the scoped reader said "ambiguous" four times where "not in the corpus" was right, and it refused three answerable questions the same way. In each case an ordinary word in the question ("long", "room", "load", "bottom") happened to sit in some other page's when-to-use phrase, so that page claimed part of the question and the rule stopped. Nothing false was returned, but the reader named contenders for questions they do not answer. Requiring an owner to declare *every* known word is strict, and one stray word is enough to break it. A majority rule is the obvious thing to try next. It was not tried here, because changing the rule after seeing these results and re-scoring the same probes would be tuning to the test.

A derived page can crowd its own source. Outside the probe set, "glaze faults" is ambiguous between the faults page and the ratified quick-reference card derived from it. A derivation that redeclares its parent's scope takes away its parent's ownership. This is recorded as FM-C4.

8.4 The coverage dial

Coverage	scoped: correct / wrong return / wrong refusal	stripped
34%	41 / 0 / 15	22 / 3 / 31
50%	41 / 0 / 15	22 / 3 / 31
67%	40 / 0 / 16	26 / 3 / 27
100%	39 / 0 / 17	22 / 3 / 31

Across its whole range the dial moved two answers. On this corpus the ownership rule does the work and the coverage constant is nearly idle. It is kept because a corpus with broader declarations may need it, and that is a guess.

8.5 What this evidence is not

- **It is one author's run.** The corpus, the declared words, the probes and the reader were written by one hand in one sitting, by a language model working from the owner's formula. The hash on the probe file shows the probes were not edited after the run. It cannot show they were written blind. This is the weakest class of evidence in MPLPB-COST-012 section 4.1.
- **The vocabulary confound of MPLPB-SWARM-013 section 13.6 applies in full.** The 100% on declared-word questions shows that pages are findable by the words their author predicted. Whether real askers use those words is not established. The 0% on prose questions is the same fact from the other side.
- **Three classes are circular** for at least one arm: they remove a field and then check for it. They are in the table for completeness and are excluded from the last row.
- **It is small.** 56 questions over 25 pages. Each percentage in the table moves by ten points or more on a single probe.
- **It is retrieval, not reconstruction.** Whether a reasoner given these answers rebuilds the corpus's rules correctly is the ablation MPLPB-LOCAL-008 section 14 named, and it is still open.

9. The Parts, as Run

Every vendored part was run on Linux with Python 3.12 from this repository's `parts/` directory. `tools/check_parts.py` repeats it.

Part	Its paper says	Ran here
Smart Local	52 tests	52 pass
Swarm	144 tests	145 pass
Networked	61 tests	61 pass
Image v2	not stated	93 pass
Local Mirror	no test suite	its own validator exits clean
Combined (this package)		93 pass

Three things were changed in the vendored copies, and nothing else.

- 1 In Smart Local and the Local Mirror, the directories `Site`, `Tests`, `Tools` and `Docs` were renamed to lower case, which is what their READMEs and imports already say. As published, Smart Local's tests do not start on a case-sensitive filesystem.
- 2 The Swarm copy is taken from the archive bundled in its repository, not from the repository tree. The tree is missing `ablation/v2/arm-structured/deploy/_index.html` and carries a stray `registry` folder inside `site/corpus-b`; three tests fail there and pass from the archive. The swarm paper says 144 tests; the suite has 145.
- 3 Nested archives and caches were removed.

The example sites of the Local Mirror, Smart Local, Swarm (both corpora) and Networked were each copied, given hashes with seal, and validated in the combined format with no errors. That is the evidence that the format in section 2 is a superset of what the family already writes.

10. Failure Modes

The FM-L and FM-S series stand. These are the ones this consolidation adds or sharpens.

FM-C1 — Unpredicted vocabulary. A page exists, answers the question, and is not returned, because the asker used words from its prose and none from its declarations. Detection: log refused questions and read them. Section 8.3 is this failure at 8 of 8.

FM-C2 — Incidental claim. An ordinary word in a when-to-use phrase gives its page a claim on questions it has nothing to do with, and the reader says ambiguous where a return or a plain not-in-corpus was right. Nothing false is served; the asker is sent to pages that do not help. Detection: ambiguous answers whose candidates each matched a different single word. Mitigation: write trigger phrases with as few ordinary words as they can carry.

FM-C3 — Status drift. A page named in supersedes still says current. The reader is unaffected. Detection: check C9. It marks a revision that did not finish.

FM-C4 — Derivative crowding. A derived or ratified page redeclares its parent's scope and the parent stops being the sole owner. Detection: ambiguity between a page and its own descendant. Mitigation: a derived page declares what it adds, not what it summarises.

FM-C5 — Self-consistent forgery. Someone edits a page and recomputes its hash. The page verifies. Its children's pins do not, so the forgery is caught only if the page has descendants or the log is intact. A leaf page edited together with the log is not caught by anything in this package. Signatures are the remedy and live in Networked MPLPB.

FM-C6 — Declared origin. A machine page marked human, or a name typed into `ratified-by` unread. Depth then counts nothing. Not detectable from the files, as the swarm paper already says of FM-S9 and FM-S12.

FM-C7 — Hub blind spot, traded for a fan-out. A hub whose pointers declare too little would refuse questions its corpora own. The fan-out in section 6 removes that at the price of asking every corpus, which

is linear in the number of pointers and makes cross-corpus ambiguity more frequent.

11. Falsifiers

11.1 — No blend (run; passes). No code path returns text that is not one page's own. Tests assert it for the three cases, the split case, hubs and fan-out.

11.2 — No erase (run; passes). After a revision the old page is present, retired, and verifies against the hash it had when current. A crash between write and retire still reads correctly.

11.3 — Count the generations (run; passes). Derivation adds one whoever derives; a revision cannot lower depth; an understated depth is reported and not believed; no question reaches a page over a profile's limit.

11.4 — The kill test (run once; mixed). Section 8. It supports "declared scope prevents wrong returns" on this corpus and contradicts "stripping scope removes the refusal" as literally stated. It needs repeating with probes written by someone who has not read the corpus, which is the first item of MPLPB-SWARM-013 section 13.5 and still needs one other person and an afternoon.

11.5 — Real vocabulary (open). Take refused questions from real use of a real corpus and count how many had an answer on a page. If FM-C1 is as common in use as it was in section 8, declared-only ownership is too strict to deploy without a prose fallback, and the fallback brings back the wrong returns it was built to prevent.

11.6 — Scale (open). The reader parses every page on every question. That is immediate at 25 pages and has not been measured at 25,000.

11.7 — Old hardware (open). The claim that a 2010 machine can run this is an inference from what the code uses: Python 3.8 syntax, the standard library, no index held in memory. It was run only on current hardware under Python 3.12.

12. What This Does and Does Not Establish

It does not establish:

- that a returned page is true;
- that declared scope beats flat storage in general. On the 42 non-circular probes the margin was 64.3% against 45.2%, on one corpus, by one author;
- that people ask in the words pages declare;
- that the format is tamper-proof. It is tamper-evident for pages with descendants or an intact log, and nothing more;
- that the formula holds on the public web, where the reader is not yours;
- that this package replaces any of the parts.

It does establish:

- that the formula can be implemented exactly, with ownership defined on sets and no score, in about twelve hundred lines with no dependency;
- that one format reads what four earlier implementations write;
- that revision can be made safe against interruption by ordering two steps and by letting the reader trust supersedes over status;
- that depth can be recomputed from lineage, so that the number in a citation is derived and not merely declared, up to the honesty of origin;
- that on the corpus tested, the reader with declared scope made no wrong return in 56 questions, and paid for it by refusing every question asked from prose.

13. Conclusion

The parts were written one at a time, each for the scale in front of it, and each is right at that scale. Underneath them is a rule short enough to say in one breath: one owner returns, two owners stop, no owner refuses, a change adds a page, a derivation adds one to a counter.

Written down as code, the rule turns out to need very little. It needs a definition of ownership that is a set comparison and not a ranking. It needs the hash to leave out the one field that has to change. It needs the new page written before the old one is retired. It needs a revision to inherit depth from what it replaces. None of that is interesting, which the swarm paper already identified as the intended outcome.

The test the formula asked for came back with a result worth more than the one expected. Stripping the fields did not remove the refusal. It removed the reader's ability to refuse the right things. With the fields, nothing wrong was returned and too much was withheld. Without them, either nearly everything was withheld or nearly everything was returned. The declared scope is not the refusal. It is what lets a refusal be exact.

Whether exact is close enough depends on whether people ask in the words authors predict, and nobody has measured that.

Appendix A — Commands

Each is run as `python3 -m mplpb_combined <command>`.

```
ask      ROOT "question" [--profile lab|internal|external] [--why] [--json]
validate ROOT [--strict]
list     ROOT [--all]
show     ROOT ID
history  ROOT ID
write    ROOT --title T --scope S [--when-to-use W] [--body B] [--prefix P]
revise   ROOT ID [--body B] [--scope S] [--ratified-by NAME]
derive   ROOT --from ID [--from ID] --title T --scope S
ratify   ROOT ID --who NAME
withdraw ROOT ID
seal     ROOT
index    ROOT
killtest ROOT PROBES.json [--out results.json]
```

ask exits 0 for a return, 2 for ambiguous, 3 for not in corpus. validate exits 1 on any error.

Appendix B — A Page

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Bisque firing schedule</title>
<meta name="mplpb:document-id" content="STUDIO-0002">
<meta name="mplpb:scope" content="Bisque firing schedule for greenware">
<meta name="mplpb:when-to-use" content="first firing; how fast to heat raw clay">
<meta name="mplpb:status" content="current">
<meta name="mplpb:hash" content="sha256:...">
<meta name="mplpb:origin" content="human">
<meta name="mplpb:origin-depth" content="0">
<meta name="mplpb:supersedes" content="STUDIO-0001@sha256:...">
</head>
<body>
<h1>Bisque firing schedule</h1>
<p>Load bone-dry ware only. Climb 100 C per hour to 600 C ...</p>
</body>
</html>
```

Appendix C — Validation Checks

Check	Level	What it finds
C1	error	A required field is missing
C2	error	Content does not match its hash
C3	error	Two pages hold one id
C4	error	Status or origin has an unknown value
C5	error	A reference names a page not in the folder
C6	error	A pinned parent no longer has the pinned hash
C7	warning	A reference carries no hash
C8	error	Declared depth disagrees with lineage
C9	warning	A superseded page still says current
C10	error	Two current pages supersede the same predecessor
C11	error or warning	A pointer has no target, or its target is not reachable
C12	error	Lineage loops
C13	warning	A file under the root is not a record
C14	error or warning	The log chain is broken, or a page is not in the log

Appendix D — Revision Note

Two changes were made after the first run of the kill test. The first is recorded because it changed a headline number. The summary row originally left out classes where the control arm scored 100%, following the ceiling rule in MPLPB-SWARM-013 Appendix A. Here that rule would have dropped the one class the scoped reader loses outright, and reported 75.0% against 17.9%. The row now excludes only circular classes and reads 64.3% against 45.2%. The second was a tokeniser fix: a possessive 's was being counted as a content word. Correcting it left every figure in section 8.2 as it was and moved one cell of the control arm in section 8.4, from 25 / 3 / 28 to 26 / 3 / 27. The probes were not touched; their hash is unchanged.

Source Note

Code is MIT. Documentation is CC BY 4.0. Each part under parts/ keeps its own licence file.

This paper and the `mplpb_combined` package were drafted with Claude (Anthropic) from the owner's formula, the uploaded papers, and the four public repositories, on 2026-10-07. Every figure in sections 8 and 9 was produced by running the included code, and `killtest/results.json` is checked against a fresh run by the test suite. No web search was available during drafting; nothing here draws on sources beyond the supplied material.

Building the PDF needs `reportlab`. That is a documentation dependency. It is not imported by the package and is not needed to run it or its tests.

Back to. [Main Index](#) > [Specification](#) / [Architecture Sub-Index](#)